

B榜提交说明文档

1. 项目信息

项目	内容
团队名称	取名困难户
A 榜排名	第 10 名
A 榜最优总分	83.58
B 榜最优总分	81.73
B 榜最优 checkpoint	ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl
联系人	穆姿羽
微信号	mzy13292929189
联系电话	13292929189

本提交包不包含数据集。复现人员需要自行准备赛事提供的训练集和 B 榜测试 noisy 数据，并按照本文档中的相对路径说明修改数据根目录。

提交包关键内容如下：

```
code/
  B榜提交说明文档.md
  B榜提交说明文档.pdf
  REPRODUCTION_GUIDE.md
  environment.yaml
  requirements.txt
ckp/
  stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl
best_checkpoints/
stage1/
stage2/
stage3/
```

B 榜最优结果对应信息：

```
score      : 81.73
checkpoint : ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl
entry      : stage3/final_inference.py
```

2. 项目概述

本项目面向点云降噪任务：输入受噪声污染的三维点云，模型预测每个点的位移向量，将偏离真实物体表面的点校正到表面附近，从而生成降噪点云。评价指标主要包括 Chamfer Distance (CD) 和 Point-to-Surface Distance (P2S)。模型需要在降低噪声的同时尽量保留物体尖锐边缘、细长结构和局部几何细节。

A 榜阶段使用 PGD (Guiding Point Cloud Denoising with Learned Structural Priors) 作为基础框架，采用两阶段级联的逐点位移预测方式：

```
noisy point cloud -> PGD1 -> intermediate point cloud -> PGD2 -> denoised point cloud
```

B 榜阶段在 A 榜代码基础上继续改动，重点围绕第二阶段法向对齐、最终推理策略和后处理进行增强，并通过分阶段续训得到 B 榜最优 checkpoint。

3. B 榜主要创新点

3.1 PGD2 引入局部法向坐标系

A 榜级联模型中，第二阶段 PGD2 直接在世界坐标系下对 PGD1 输出继续预测残余位移。B 榜版本在 PGD2 前向时加入局部 PCA 法向坐标系：

```
PGD1 output patch
-> estimate local PCA axes
-> build tangent / tangent / normal frame
-> rotate PGD1 points into local normal frame
-> PGD2 predicts residual displacement in local frame
-> rotate PGD2 displacement back to world frame
-> merge overlapping patch predictions
```

这一改动使第二阶段更明确地区分切向结构和法向噪声，有利于减少沿表面切向的错误收缩，并增强对薄片、尖角、边界和细长结构的保护。相关实现位于：

```
stage3/src/model/ops.py
stage3/final_inference.py
```

其中 `normal_alignment_frames`、`to_normal_frame`、`from_normal_frame` 负责 PCA 法向坐标系构造和坐标变换，`final_inference.py` 中的 `Predictor` 在 PGD1 和 PGD2 之间调用该逻辑。

3.2 优化最终推理流程

B 榜最终推理不再使用早期分散的预测脚本，而是整理为单入口：

```
stage3/final_inference.py
```

该脚本完成从 noisy 输入到最终提交 zip 的完整流程：

```
noisy.npy
-> overlapping patch PGD1/PGD2 forward
-> 保存 pgd1.npy、pgd2_raw.npy、原始 denoised.npy
-> local-alpha
-> Surface Gate
-> Coverage Repair
-> final denoised.npy
-> submission zip
```

最终推理使用更高覆盖率的 patch 前向和 adaptive merge：

```
patch_size           = 2600
patch_seed_k         = 52
patch_seed_k_alpha   = 10
patch_merge          = adaptive
patch_merge_beta     = 8.0
patch_adaptive_alpha_max = 1.0
patch_adaptive_tau    = 0.008
normal_frame_sign_mode = stable_dominant
tta_passes           = 1
```

adaptive merge 同时考虑最近中心的 hard 结果和距离加权平均结果。当重叠 patch 预测一致时更多采用平均以降低噪声，当不同 patch 分歧较大时退回 hard 结果以保护局部尖锐结构。

3.3 加入推理阶段后处理

B 榜最终结果在 PGD2 原始输出后加入三段后处理，均不读取测试集 GT、mesh 或外部 normal，不更新 checkpoint 参数。

Local-alpha

根据 `pgd2_raw - pgd1` 的局部一致性自适应调节 PGD2 位移强度。邻域位移方向更一致时适度增强 PGD2 修正，邻域分歧较大时降低修正强度，减少局部过冲。

```
k           = 16
base        = 1.20
delta       = 0.07
band        = [0.05, 0.25]
```

Surface Gate

在 $k=8/12/16/24/32$ 多个尺度上基于局部 PCA 法向生成候选表面修正，再用五折自训练 Gate MLP 根据局部几何特征和 PGD1/PGD2 一致性预测逐点修正强度。

```
stage3/models/surface_gate/gate_fold0.npz
stage3/models/surface_gate/gate_fold1.npz
stage3/models/surface_gate/gate_fold2.npz
stage3/models/surface_gate/gate_fold3.npz
stage3/models/surface_gate/gate_fold4.npz
gate_scale = 0.575
```

Coverage Repair

Surface Gate 后可能出现局部点分布轻微聚集，因此最后沿局部切平面进行受限重分布，改善点云覆盖性。

```
k = 16
coverage_strength = 0.90
max_step_ratio = 0.01
```

4. 相对于 A 榜算法的改动

与 A 榜提交代码相比，B 榜主要改动如下：

1. 第二阶段去噪前进行法向统一：对每个 patch 建立局部 PCA 法向坐标系并统一法向方向，使第二阶段在一致的局部几何坐标系中完成去噪。
2. 推理阶段加入后处理：在网络预测结果基础上加入 local-alpha、Surface Gate 和 Coverage Repair 等后处理，进一步保护局部结构并提升最终去噪效果。

改动可以概括为以下复现链路：

```
A榜 PGD/NAA-PGD two-stage denoise
+ PGD2 local normal-frame prediction
+ higher-overlap final inference
+ adaptive patch merge
+ local-alpha postprocess
+ Surface Gate postprocess
+ Coverage Repair postprocess
+ Stage3 staged finetuning
= B榜 81.73 with checkpoint 544
```

B 榜改动保持了 A 榜 PGD/NAA-PGD 基础网络结构和两阶段级联思路，主要提升第二阶段几何坐标表达、最终推理覆盖率和推理阶段几何修正能力。

5. 代码结构说明

清理后的提交代码目录如下：

```
code/
  REPRODUCTION_GUIDE.md
  详细复现流程文档。

  B榜提交说明文档.md
  本文档源文件。

  environment.yaml
  requirements.txt
  Conda 环境和 pip 依赖清单。

  ckp/
  关键 checkpoint。B 榜最优结果默认使用 544 权重。

  best_checkpoints/
  额外保留的 B 榜后期候选 checkpoint，用于审计和备份。

  stage1/
    run.py
    Stage1 训练入口，训练单阶段 PGD1。
    configs/
      data/train.yaml
      task/train.yaml
      task/train_from_checkpoint_349_200.yaml
      model/core.yaml
      transform/ops.yaml
      system/run.yaml
    datalist/
      train.txt、validate.txt、test.txt
    src/
      data/、model/、system/、utils/
    experiments/
      Stage1 checkpoint，保留用于检查。
      third_party/PointCloudLib/
      内置 KNN/group 算子。

  stage2/
    run.py
    Stage2 训练入口，加载 PGD1 并训练 PGD2。
    configs/
```

```
data/train.yaml
task/freeze40.yaml
model/core.yaml
transform/ops.yaml
system/run.yaml
datalist/
src/
experiments/
    Stage2 checkpoint, 保留用于检查。
third_party/PointCloudLib/

stage3/
    run.py
        Stage3 联合训练入口。
    final_inference.py
        B 榜最终推理入口。
    compare_prediction_dirs.py
复现结果对比脚本。
configs/
    task/joint.yaml
    task/joint_from_200_hard005_tail015_bs32.yaml
    task/joint_from_362_hard005_tail015_bs16_fps_np2_100.yaml
    task/joint_from_461_hard005_tail015_bs8_fps_np4_100.yaml
    transform/ops.yaml、ops_fps_np2.yaml、ops_fps_np4.yaml
    system/run.yaml
    system/run_hard005_tail015_bs32.yaml
    system/run_hard005_tail015_bs16_fps_np2_from362.yaml
    system/run_hard005_tail015_bs8_fps_np4_from461.yaml
    model/core.yaml
    data/train.yaml
datalist/
    train.txt、validate.txt、test.txt
models/surface_gate/
    Surface Gate 五折权重。
src/
    data/、model/、system/、utils/
experiments/
    Stage3 checkpoint, 保留用于检查。
third_party/PointCloudLib/
```

其中，复现 B 榜最优结果的最小核心文件包括：

```
stage3/final_inference.py
stage3/src/
stage3/configs/model/core.yaml
```

```
stage3/configs/transform/ops.yaml
stage3/third_party/PointCloudLib/
stage3/models/surface_gate/
stage3/datalist/test.txt
ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl
```

6. B 榜最优 checkpoint

B 榜最优总分为：

81.73

对应 checkpoint 为：

ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl

该 checkpoint 是联合 PGD1+PGD2 权重，包含 pgd1.* 和 pgd2.* 参数。提交包中同时保留 ckp/ 下训练链关键 checkpoint，方便复现续训过程。

7. B 榜最优结果复现方式

最小复现命令总览如下，完整说明见后续小节：

```
cd /path/to/code
conda env create -f environment.yaml
conda activate pgd_jt
python -m pip install -r requirements.txt

export TEST_ROOT=/path/to/dataset_test_noisy/dataset_test_noisy
export CUDA_VISIBLE_DEVICES=0
export cc_path="$(which g++)"
export HWLOC_COMPONENTS=-gl
export OBJ_MESH_CACHE_MB=512

python stage3/final_inference.py \
  --checkpoint ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl \
  --input-root "$TEST_ROOT" \
  --test-list stage3/datalist/test.txt \
  --work-dir stage3/final_runs/544_b_200 \
  --output-zip stage3/final_runs/result.zip \
  --gpu-id 0 \
  --post-workers 16
```

7.1 环境安装

推荐环境：

```
Ubuntu 22.04 或兼容 Linux
NVIDIA GPU, 最终单卡推理建议 24GB 显存
CUDA Toolkit 推荐 12.4
Python 3.9 或 3.10
g++、nvcc 可用
```

创建环境：

```
cd /path/to/code

conda env create -f environment.yaml
conda activate pgd_jt
python -m pip install -r requirements.txt
```

也可手动安装核心依赖：

```
conda create -n pgd_jt python=3.9 pip -y
conda activate pgd_jt
python -m pip install --upgrade pip
python -m pip install jittor==1.3.11.0 numpy==1.26.4 scipy==1.13.1 omegaconf==2.3.1
PyYAML==6.0.3 tqdm==4.68.3 astunparse==1.6.3 antlr4-python3-runtime==4.9.3 six==1.17.0
```

运行前建议设置：

```
export CUDA_VISIBLE_DEVICES=0
export cc_path="$(which g++)"
export HWLOC_COMPONENTS=-gl
export OBJ_MESH_CACHE_MB=512
```

7.2 数据路径

请勿将数据集放入提交包。复现时自行准备 B 榜测试 noisy 数据，目录格式应为：

```
<TEST_ROOT>/shapenet/<category>/<sample_id>/noisy.npy
```

stage3/datalist/test.txt 中每行是相对于 <TEST_ROOT> 的样本目录，例如：

```
shapenet/00000000/btest_000001
```

复现前设置：

```
export TEST_ROOT=/path/to/dataset_test_noisy/dataset_test_noisy
```

7.3 单卡推理命令

在代码根目录执行：

```
python stage3/final_inference.py \  
  --checkpoint ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl \  
  --input-root "$TEST_ROOT" \  
  --test-list stage3/datalist/test.txt \  
  --work-dir stage3/final_runs/544_b_200 \  
  --output-zip stage3/final_runs/result.zip \  
  --gpu-id 0 \  
  --post-workers 16
```

输出目录：

```
stage3/final_runs/544_b_200/  
  run_manifest.json  
  intermediates/<sample>/pgd1.npy  
  intermediates/<sample>/pgd2_raw.npy  
  intermediates/<sample>/denoised.npy  
  local_alpha/<sample>/denoised.npy  
  final/<sample>/denoised.npy
```

最终提交 zip：

```
stage3/final_runs/result.zip
```

zip 内目录格式为：

```
shapenet/<category>/<sample_id>/denoised.npy
```

提交前检查：

```
unzip -tq stage3/final_runs/result.zip
```

7.4 断点续跑

`final_inference.py` 默认不覆盖已有完整文件。推理中断后重新执行同一命令即可续跑。若需要全部重跑，加：

```
--overwrite
```

也可以分阶段运行：

```
python stage3/final_inference.py ... --mode forward
python stage3/final_inference.py ... --mode postprocess
python stage3/final_inference.py ... --mode package
```

8. 训练复现方式

完整训练/续训链如下：

1. Stage1: A 榜数据训练 500 轮
2. Stage1: 从第 349 轮 checkpoint 在 A+B 完整数据上续训 200 轮
3. Stage2: 加载 Stage1 199.pkl, 训练 PGD2 40 轮
4. Stage3: 加载 Stage1 199.pkl + Stage2 39.pkl, 联合训练到 joint_200.pkl
5. Stage3: 从 joint_200.pkl 使用 hard005 tail015 bs32 续训, 选择 362 checkpoint
6. Stage3: 从 362.pkl 改为 num_patches=2、bs16 续训到 461.pkl
7. Stage3: 从 461.pkl 改为 num_patches=4、bs8 续训到最终 544.pkl

完整训练命令链如下：

```
python stage1/run.py --task stage1/configs/task/train
python stage1/run.py --task stage1/configs/task/train_from_checkpoint_349_200
python stage2/run.py --task stage2/configs/task/freeze40
python stage3/run.py --task stage3/configs/task/joint
python stage3/run.py --task stage3/configs/task/joint_from_200_hard005_tail015_bs32
python stage3/run.py --task stage3/configs/task/
joint_from_362_hard005_tail015_bs16_fps_np2_100
python stage3/run.py --task stage3/configs/task/
joint_from_461_hard005_tail015_bs8_fps_np4_100
```

训练数据根目录需要在以下配置中修改：

```
stage1/configs/data/train.yaml
stage2/configs/data/train.yaml
stage3/configs/data/train.yaml
```

将 `input_dataset_dir` 改为实际训练集路径：

```
datapath:
  input_dataset_dir: /path/to/dataset_train
```

8.1 Stage1 基础训练

```
python stage1/run.py --task stage1/configs/task/train
```

输出：

```
stage1/experiments/stage1_checkpoint_<epoch>.pkl
```

后续使用第 349 轮 checkpoint。

8.2 Stage1 A+B 续训

```
python stage1/run.py --task stage1/configs/task/train_from_checkpoint_349_200
```

关键配置：

```
load_ckpt: ckp/stage1_checkpoint_349.pkl
trainer:
  epochs: 200
```

后续使用：

```
stage1/experiments/stage1_checkpoint_199.pkl
```

8.3 Stage2 训练 PGD2

```
python stage2/run.py --task stage2/configs/task/freeze40
```

关键配置：

```
stage1_ckpt: ckp/stage1_checkpoint_199.pkl
epochs: 40
lr: 0.00005
lambda_hard: 0.002
hard_tail_ratio: 0.1
```

后续使用：

```
stage2/experiments/stage2_checkpoint_39.pkl
```

8.4 Stage3 联合训练到 joint_200

```
python stage3/run.py --task stage3/configs/task/joint
```

关键配置：

```
stage1_ckpt: ckp/stage1_checkpoint_199.pkl
stage2_ckpt: ckp/stage2_checkpoint_39.pkl
joint_epochs: 200
joint_batch_size: 16
stage2_lr: 0.00005
stage1_lr_mult: 0.1
stage1_loss_weight: 0.1
lambda_hard: 0.002
hard_tail_ratio: 0.1
```

输出：

```
stage3/experiments/stage3_checkpoint_joint_200.pkl
```

8.5 Stage3 hard005 tail015 bs32 续训

```
python stage3/run.py --task stage3/configs/task/joint_from_200_hard005_tail015_bs32
```

关键配置：

```
init_joint_ckpt: ckp/stage3_checkpoint_joint_200.pkl
start_epoch: 201
joint_epochs: 400
joint_batch_size: 32
lambda_hard: 0.005
hard_tail_ratio: 0.15
```

后续使用：

```
ckp/stage3_checkpoint_hard005_tail015_bs32_joint_362.pkl
```

8.6 Stage3 num_patches=2、bs16 续训

```
python stage3/run.py --task stage3/configs/task/
joint_from_362_hard005_tail015_bs16_fps_np2_100
```

关键配置：

```
init_joint_ckpt: ckp/stage3_checkpoint_hard005_tail015_bs32_joint_362.pkl
start_epoch: 363
joint_epochs: 462
joint_batch_size: 16
components:
  transform: stage3/configs/transform/ops_fps_np2
```

ops_fps_np2.yaml 中训练 patch 数：

```
num_patches: 2
```

后续使用：

```
ckp/stage3_checkpoint_hard005_tail015_bs16_fps_np2_from362_joint_461.pkl
```

8.7 Stage3 num_patches=4、bs8 续训

```
python stage3/run.py --task stage3/configs/task/
joint_from_461_hard005_tail015_bs8_fps_np4_100
```

关键配置：

```
init_joint_ckpt: ckp/stage3_checkpoint_hard005_tail015_bs16_fps_np2_from362_joint_461.pkl
start_epoch: 462
joint_epochs: 561
joint_batch_size: 8
stage2_lr: 0.00003
stage1_lr_mult: 0.05
stage1_loss_weight: 0.15
lambda_hard: 0.005
hard_tail_ratio: 0.15
components:
  transform: stage3/configs/transform/ops_fps_np4
```

ops_fps_np4.yaml 中训练 patch 数：

```
num_patches: 4
```

最终 B 榜最优推理使用：

```
ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl
```

9. 注意事项

1. 提交包不包含数据集，复现时需自行准备赛事数据。
2. B 榜最优结果默认使用 `ckp/stage3_checkpoint_hard005_tail015_bs8_fps_np4_from461_joint_544.pkl`。
3. 最终推理不读取测试集 GT、mesh 或外部 normal。
4. `--post-workers` 是 CPU 后处理进程数，不是 GPU worker 数。
5. 24GB 显存下，最终推理建议一张卡只启动一个 `final_inference.py`。
6. 不要修改最终推理 patch 和后处理参数，否则不再是 B 榜 81.73 对应策略。
7. zip 内应直接包含 `shapenet/<category>/<sample_id>/denoised.npy`，不要多包一层 `final/` 或 `stage3/final_runs/`。